Lecture-3

Understanding Processes in an OS

Learning Objectives

- Understand what a process is in an OS.
- Learn how processes are created, scheduled, and terminated.
- Explore process states and transitions.
- Understand scheduling algorithms and context switching.
- Learn Inter-Process Communication (IPC) techniques.

What is a Process?

A process is a program in execution, consisting of program code and its activity.

Key Features:

- Has a unique Process ID (PID).
- Consumes system resources (CPU, memory, I/O).
- Can run independently or interact with other processes.

Example: Opening a web browser starts multiple processes (main browser, rendering engine, network requests).

Task

Use Task Manager (Windows) or `ps -aux` (Linux) to view running processes and their resource usage.

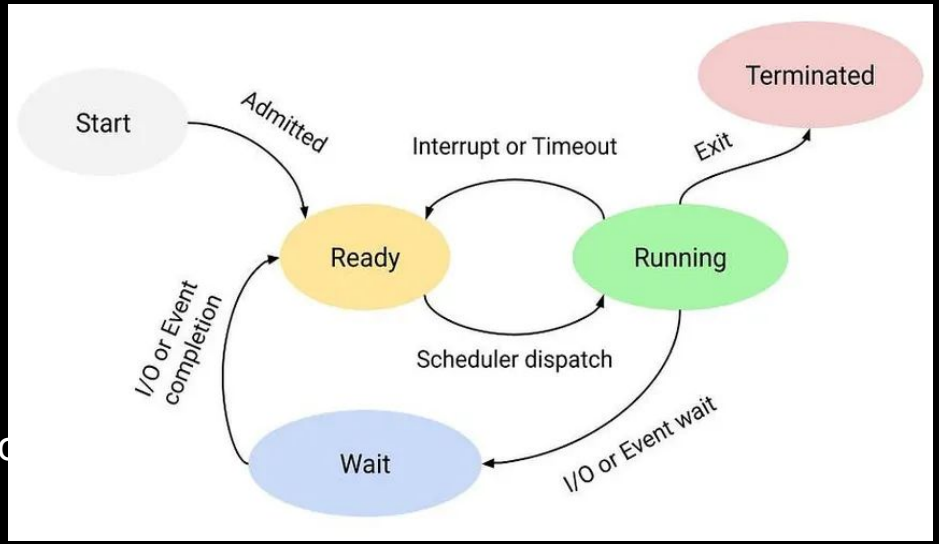
Process States

A process moves through different states during its lifecycle.

Main States:

- Start: Process is being created.
- Ready: Waiting for CPU time.
- Running: Currently executing on CPU.
- Wait: Waiting for an event (e.g., I/O completio
- Terminated: Finished execution or forcefully stopped.

Example: A video editing software may be in a "Wait" state while rendering video frames.



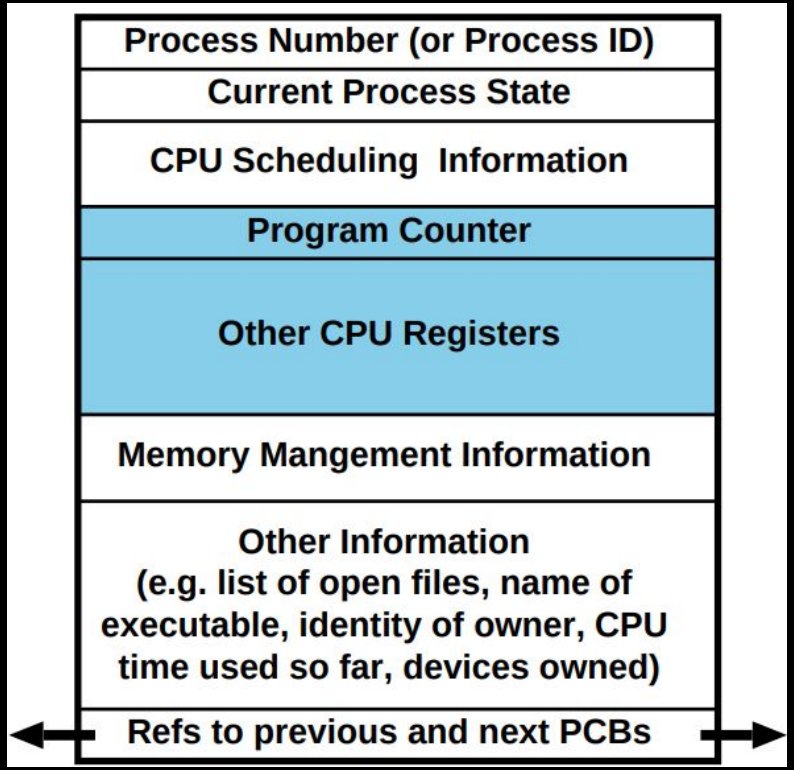
Process Control Block (PCB)

OS maintains information about every process in a data structure called a *PCB*.

Contains:

- Process ID (PID)
- Program Counter (PC)
- CPU registers
- Scheduling information
- Memory management info, and more.

Example: The OS uses PCBs to track background services like antivirus scans.



Task

Explore process information using `cat /proc/<PID>/status` (Linux) or Process Explorer (Windows).

Process Creation & Termination

New processes are created using system calls and terminated after execution.

Process Creation:

- Parent process spawns child processes.
- Uses fork() (Linux) or CreateProcess() (Windows).

Process Termination:

- Exits typically or is killed due to an error.

Example: A mobile app launching background processes for notifications.

Task

Write a simple C program using `fork()` or `CreateProcess()` and observe the output.

A simple process creation example using the fork() system call.

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();  // Create a child process
    if (pid == 0) {
        printf("Child process: %d\n", getpid());
    } else {
        printf("Parent process: %d\n", getpid());
    }
    return 0;
}
```

Explanation

1. **Headers Included**
   - `#include <stdio.h>` → Standard Input/Output functions (`printf`).
   - `#include <unistd.h>` → Provides system calls like `fork()`.
2. **`fork()` System Call**
   - `fork()` creates a new child process.
   - It **returns the child's process ID (PID) to the parent**.
   - It **returns 0 to the child process**.
3. **Checking `pid`**
   - If `pid == 0`, the process is the **child**, so it prints:
   - Child process: <child_PID>
   - If pid > 0, the process is the parent, so it prints:
   - Parent process: <parent_PID>
4. **Two Separate Processes**
   - Both parent and child execute the code **after `fork()`** independently.
   - The child gets a copy of the parent's memory but runs separately.

Expected Output:

- The parent's PID is 12345.
- The child's PID is 12346

Task

Write a program where the parent waits for the child to finish.

Process Scheduling

The OS determines which process runs on the CPU at a given time.

Types of Scheduling:

- Long-term scheduler: Determines which jobs enter the ready queue.
- Short-term scheduler: Selects which process gets CPU time.
- Medium-term scheduler: Swaps processes in/out of memory.

Example: The CPU constantly switches between active applications like Zoom and a web browser.

Task

Check CPU scheduling using htop (Linux) or Task Manager (Windows).

CPU Scheduling Algorithms

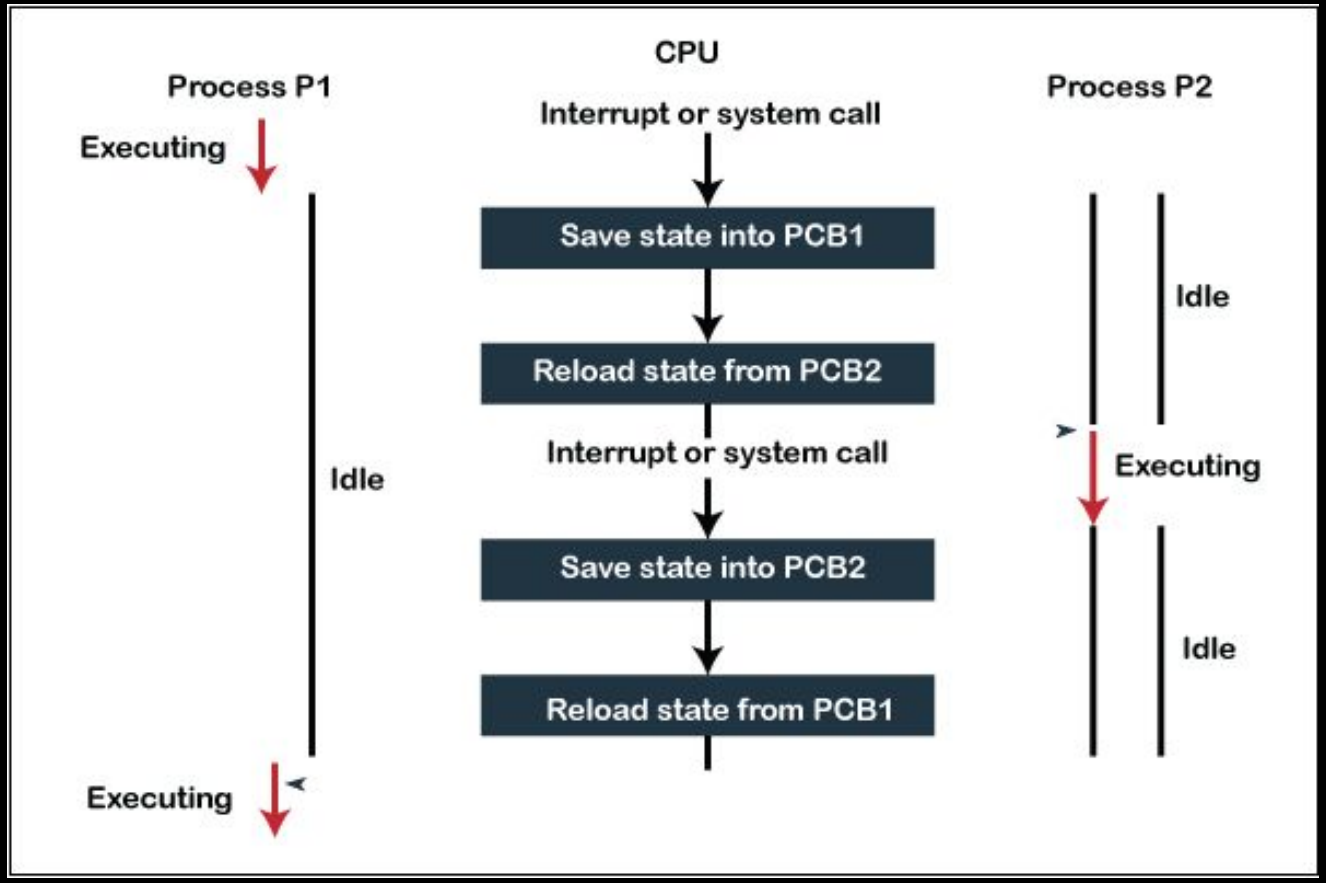
Different methods for deciding process execution order.

Main Algorithms:

- First Come, First Served (FCFS): Processes executed in order of arrival.
- Shortest Job Next (SJN): Process with the shortest execution time runs first.
- Round Robin (RR): Each process gets a fixed CPU time slice.
- Priority Scheduling: Higher-priority processes run first.

Example: Online ticket booking systems use Priority Scheduling for VIP customers.

Context Switching



Context Switching

The OS switches the CPU between processes to enable multitasking.

Machine environment during the time the process is actively using the CPU.

- To switch between processes, the OS must:
  - save the context of the currently executing process (if any), and
  - restore the context of that being resumed.

Example: Switching between a video call and a document editor

Task

Monitor CPU context switches using vmstat 1 (Linux) or Performance Monitor (Windows).

Inter-Process Communication (IPC)

Mechanisms for processes to exchange data.

Types of IPC:

- Shared Memory: Processes access the same memory space.
- Message Passing: Processes send and receive messages.

Example: A chat application where the frontend communicates with the backend via sockets.

Task

Write a basic IPC program using shared memory in C.

Hands-on Activity

Activity:

- Open multiple applications and observe their process behavior.
- Use ps, top, and kill commands (Linux) or Task Manager or Resource Monitor (Windows).

Objective: Understand how the OS manages processes.

Group Discussion

Topics for Discussion:

- How do modern operating systems handle process scheduling efficiently?
- What are the different CPU scheduling algorithms?
- Can you think of any applications where multithreading is essential?
- How does process management impact system performance in high-load environments?
- Compare real-world scenarios where process and thread optimization is critical (e.g., gaming, web servers, mobile apps).

Summary…